Clinic Phone Agent — 小诊所 AI 电话前台（骨架）

接电话 → 回答诊所常见问题 → 预约 / 改约 / 取消 → 处理不了的留言或转真人 → 通话结束后写摘要给前台。

> 这是一个**能跑、带测试的骨架**，没做优化。先读第 1–3 节理解"这事行业里怎么做"，再看第 4 节怎么跑。

---

1. 行业里一般怎么做（三条路）

这个问题定义得很清楚，行业里基本就三种做法：

路线	代表	你要做的事	适合
A. 直接买成品	各种 "AI receptionist for dental/medical" 的垂直 SaaS	配置 + 对接	诊所自己用，不想要技术团队
B. 托管语音 Agent 平台	Vapi、Retell AI、Bland、Synthflow	写 prompt、把"查空档 / 预约"做成 webhook 给平台调用	代理公司 / 外包最常见，上线最快
C. 自己搭	Twilio（电话）+ OpenAI Realtime（语音模型），或者 Pipecat / LiveKit Agents 框架	自己负责音频桥接、工具调用、日志	想掌控数据、成本、合规，或者要做成产品

不管选哪条，**架构其实都一样**，只是哪层是你写、哪层是买：

```
电话网络（PSTN）
   │
电话运营商 / 号码（Twilio）          ← B 路线里平台帮你包了
   │  音频流
语音模型（speech-to-speech：OpenAI Realtime；
         或者三段式：STT → LLM → TTS）        ← B 路线里平台帮你包了
   │  工具调用（function calling）
你的业务后端：排班规则、患者、预约、留言    ← 不管哪条路线，这层都得你写
   │
诊所现有系统（EHR / 预约软件）或者我们自己的 DB
```

这个仓库走的是 C 路线，理由如下：

- 你已经用过 OpenAI Realtime，最熟。

- 面试要的是"能不能做"，自己搭能证明你理解每一层。之后真要快速交付，换成 B（Vapi/Retell）也只是把"语音那层"换掉，`app/scheduling.py` + `app/tools.py` **这层原封不动**——它们本来就是 webhook 能直接调的普通函数。
- 体量小：一两百个病人，一天大概十几到几十通电话。**一台小服务器 + SQLite/Postgres 就够了**，不需要队列、K8s、微服务。

关于"一两百个客户"——面试时一定要问清楚

- 如果是**一家诊所有一两百个病人** → 就是这个仓库现在的单租户形态。
- 如果是**这家公司服务一两百家诊所** → 要做多租户：`clinic.json` 改成数据库里的 `clinics` 表，按被叫号码（Twilio 的 `To` 参数）找到对应的诊所配置/prompt/数据。代码改动不大，但架构讨论的重点就不一样了。

---

2. 关于 RAG：这个体量不需要

小诊所的"知识"就是地址、停车、营业时间、取消政策、保险……几页纸，几千字。 **直接全塞进 system prompt** 比 RAG 更好：没有检索延迟（电话里每 300ms 都能听出来）、不会漏检、prompt cache 友好。

`app/knowledge.py` 已经做了自动切换：知识库 < 12k 字符时内联进 prompt；超过以后自动给模型加一个 `search_clinic_info` 工具（用的是很小的 BM25，零依赖）。真到几百份文档再换 embedding + pgvector。

真正需要"检索"的是结构化数据——空档、预约、患者——这些靠**工具调用查数据库**，不是 RAG。

---

3. 架构和关键设计（面试时讲这些）

```
来电 ──> Twilio 号码 ──POST /twilio/voice──> 返回 TwiML <Connect><Stream>
                │
                └── wss /twilio/media-stream  (G.711 μ-law 8kHz)
                          │   app/telephony/twilio_stream.py  (音频桥 + 打断处理)
                          │
                          └── wss OpenAI Realtime  (app/realtime_agent.py：会话配置、工具循
环、转写)
                                    │ function call
                                    v
                            app/tools.py  (工具定义 + 分发，身份绑定在通话上下文)
                                    │
                            app/scheduling.py  (纯业务规则，不碰 LLM)
                                    │
                            app/db.py  (SQLite → Postgres)
通话结束 ──> app/call_lifecycle.py：存转写 + 工具调用记录 + LLM 摘要 → /admin/calls 给前台看
```

设计原则

1. **模型只"提议"，代码来"执行"。** 营业时间、提前量、防重复预约、医生和服务是否匹配，全都在 `scheduling.py` 里硬校验。模型说错时间也订不进去。写入时会再查一次冲突（slot 可能刚被别人占了）。
2. **身份绑定在通话上，不在参数里。** `verify_patient`（姓名 + 生日）成功后，patient_id 存在 `CallContext` 里；工具参数里**根本没有 patient_id**，所以来电者再怎么"说服"模型，也动不了别人的预约。
3. **永远有退路。** 处理不了 → `take_message` 留言；客户要求 / 情绪不好 / 卡住两次 → `transfer_to_human` 转前台；急症 → 让对方挂了打 911（写在 prompt 安全规则最前面）。
4. **不给医疗建议，不报检查结果。**
5. **说完再挂。** `end_call` / 转接不会立刻执行：等模型把"再见"说完，Twilio 回传 `mark` 事件确认音频播完了，才真正挂断或转接。
6. **打断（barge-in）。** 来电者一开口：清掉 Twilio 里还没播的音频 + `conversation.item.truncate` 告诉模型"你只说到这里"，模型就不会以为对方听到了没播出来的部分。
7. **文字和语音用同一套工具。** `scripts/text_chat.py` 用同样的 prompt、工具和数据库，打字就能测完整业务流程，不用每次都打电话。

两种电话接入方式（都已实现）

- **默认：Twilio Media Streams**（`twilio_stream.py`）——音频经过我们的服务器。可控、可调试，资料最多。
- **可选：OpenAI SIP**（`openai_sip.py`）——Twilio Elastic SIP Trunk 直接连到 `sip:<project_id>@sip.api.openai.com`，OpenAI 发 `realtime.call.incoming` webhook，我们

accept 后挂一个 sideband websocket 只处理工具调用。少一跳，延迟更低，但跟 OpenAI 绑得更紧。

---

4. 怎么跑

```
uv sync
```

```
cp .env.example .env
```

填好 `OPENAI_API_KEY`（Twilio 那几项先不填也行）。

```
uv run python -m scripts.seed
```

不用电话，先打字测：

```
uv run python -m scripts.text_chat
```

可以试："我是 Alice Nguyen，生日 1985-04-12，想把明天的预约改到下周三下午。"

跑测试：

```
uv run pytest -q
```

接真电话：

1. 启动服务： `bash uv run uvicorn app.main:app --port 8000`
2. 暴露公网 https，然后把地址填进 `.env` 的 `PUBLIC_BASE_URL`： `bash ngrok http 8000`
3. Twilio 控制台 → 买一个号码 → Voice "A call comes in" → Webhook `POST https://<你的域名>/twilio/voice`。
4. 打这个号码。

前台接口（`Authorization: Bearer <ADMIN_TOKEN>`）：`GET /admin/appointments?day=YYYY-MM-DD`、`/admin/messages`、`/admin/calls`。

---

5. 代码地图

文件	作用
`clinic.json`	诊所配置：时区、营业时间、休息日、服务项目和时长、医生、转接号码
`knowledge/*.md`	FAQ / 政策（诊所直接改 markdown 就行）

文件	作用
`app/scheduling.py`	排班规则：找空档、预约、改约、取消、患者查找/注册
`app/tools.py`	给模型的 10 个工具 + 分发器 + `CallContext`
`app/prompts.py`	system prompt（角色、语气、安全规则、流程，每通电话注入当前时间和来电号码）
`app/realtime_agent.py`	跟传输方式无关的 Realtime 会话处理：配置、工具循环、转写
`app/telephony/twilio_stream.py`	Twilio ⇄ OpenAI 音频桥、打断、说完再挂、签名校验
`app/telephony/openai_sip.py`	SIP 直连方式
`app/call_lifecycle.py`	通话记录落库 + 结束后 LLM 摘要
`app/admin.py`	前台查看接口
`scripts/text_chat.py`	打字测试
`tests/`	排班规则、工具权限、TwiML，还有一个用假 OpenAI 服务的端到端桥接测试

---

6. 模拟电话实测（2026-10-17，真的 OpenAI Realtime）

`scripts/simulate_call.py`：用"假来电者"（LLM 扮演角色 + TTS 合成语音）代替 Twilio，以真实电话的格式（8kHz μ-law、实时推流、回传播放标记）打到 `/twilio/media-stream`。服务器那边走的是生产代码。每通电话在 `data/sim_calls/<场景>/` 下输出 `transcript.md` 和双声道录音 `call.wav`。

```
uv run python -m scripts.simulate_call
```

一共跑了 4 轮，前后找到并修掉的问题：

问题	原因	修法
来电者说完以后要等 4–8 秒 agent 才开口	`semantic_vad` 在电话音质下判断"说完了"太慢	改用 `server_vad` 600ms，延迟稳定在 1.1–2 秒；可通过 `VAD_TYPE` 配置
诊所下班了还"转真人"（转过去会没人接）	转接前没检查营业时间	`transfer_to_human` 在代码里检查营业时间，关门时让 agent 改为留言
新患者被订给了不收新病人的 Chen 医生	这条规则只写在 FAQ 里	`clinic.json` 增加 `accepting_new_patients`，查空档和预约时由代码强制
续药留言没有生日，前台找不到病历	只靠 prompt 约束，模型时灵时不灵	`take_message` 增加 `category`；续药、账单、检查结果三类没给生日就拒绝保存
中文姓名：丽 / 李 / 力 分不清，登记成了"李王"	电话里同音字没法区分	改问健康卡上的英文名，逐个字母复述

问题	原因	修法
中文场景把 10 月说成 11 月	模型翻译英文月份时口误	工具同时返回数字日期 `date`
问候语没有诊所名字；一个回合里说三遍"稍等"；念出 prompt 规则	prompt 写法	改 prompt
订完直接挂线，没问"还有别的吗"	prompt 写法	改 prompt：先问还有没有事，说完再见立刻调用 `end_call`
先说"正在为您转接"，查完又改口"诊所关门了"	prompt 写法	工具成功之后才宣布转接

多语言（2026-10-18）

示例诊所的医生是编的：沈伟医生（英语、普通话、粤语）、Patel 医生（只会英语，不收新病人）、黄护士（英语、粤语），前台会英语和普通话。换成真实名单时只需要改 `clinic.json`：`name_zh`、`aliases`、`languages`、`front_desk_languages`。

场景	结果	发现的问题 → 修法
普通话新患者点名"沈医生"	✅ 预约成功，前台收到英文摘要	原来按英文名匹配，"沈医生"找不到 → `clinic.json` 增加 `aliases`，代码同时匹配中英文名
中英夹杂改约（老病人 Mei Lin）	✅ 第二次跑通过	第一次 Mei 被听成 May，身份核对失败 → 姓 + 生日 + 来电号码一致时允许名字拼写有出入；姓名颠倒也能认出。生日不对永远不放行
粤语新患者打流感针	✅ 第二次全程粤语	第一次回答夹杂普通话 → prompt 明确"粤语不是普通话"
英文点名不收新病人的 Patel 医生	✅ 代码挡住，改推沈医生	—
中文病人一接通先听到英文	✅	改为中英双语问候

另外：给前台的摘要和留言内容一律用英文，并注明来电者说的是什么语言。 仍要注意：Realtime 的粤语明显比普通话弱，粤语病人多的话要用真人重点测；通话末尾偶尔会切回英文。

还没解决 / 要注意的：

- 合成语音太干净了。真实电话有噪音、口音、对方中途停顿，ASR 准确率和 VAD 误判一定比这里差。上线前必须用真号码、真人测。
- 一个回合里连着调用多个工具（核对身份 → 注册 → 查空档），每多一轮大约多 1 秒，最长 3–4 秒。可以合并工具或让模型并行调用。
- 中文语音偶尔有用词错误，比如把"疫苗"说成别的词（也可能是测试脚本自己的语音识别听错了）。
- 只靠 prompt 的规则是不稳定的；凡是必须做到的，都要写进代码。

7. 真正难的地方（骨架里还没做，面试时主动提）

1. **对接诊所现有的预约系统**——这是最大的未知数，也最容易超工期。诊所多半已经在用某个 EHR / 预约软件（加拿大常见 Jane App、OSCAR、Accuro；美国常见 Athena、eClinicalWorks、Dentrix……）。要么它有 API（把 `scheduling.py` 的 DB 读写换成调它的 API），要么没有（那就是我们的 DB 当主数据，前台在 `/admin` 上看，或者做同步）。**第一个要问的问题。**
2. **合规。** 美国：HIPAA，需要跟 OpenAI、Twilio 都签 BAA（两家都有，但要走企业流程）；加拿大安省：PHIPA。通话录音和转写都是 PHI：要加密存储、设保留期限、控制访问权限；录音前还要告知对方。
3. **姓名 / 生日的识别准确率。** 电话音质 8kHz，名字很容易听错。常见做法：让对方拼一下、复述确认，或者先用来电号码匹配候选人，再用生日确认。
4. **延迟。** 目标是对方说完后 < 1 秒开始回应。speech-to-speech（Realtime）在这点上比 STT→LLM→TTS 三段式有优势；服务器尽量部署在离 OpenAI / Twilio 近的区域（美东）。
5. **评测。** 准备一套场景脚本（预约、改约、听错名字、问医疗问题、急症、要找真人、说中文……），用 `text_chat` 的逻辑做自动回归测试；上线后每周抽查通话摘要。
6. **生产化。** SQLite → Postgres，并加排他约束保证不会重复预约；`/admin` 换成真正的登录；部署（Fly.io / Render / AWS 一台小机器就够）；监控 Twilio 和 OpenAI 的错误率。
7. **外呼**（预约提醒、确认）——后续很自然的扩展，用 Twilio 发起外呼，接到同一套 agent 上。

8. 面试时可以问客户的问题

- 现在用什么预约 / 病历系统？有没有 API？
- 一天大概多少通电话？高峰在什么时候？下班后的电话现在怎么处理？
- 来电里预约、改约、问问题、要处方续药的比例大概是多少？
- 需要哪些语言？
- 在美国还是加拿大？对合规（BAA / 录音告知）有什么要求？
- "一两百个客户"指的是病人，还是你们服务的诊所数量？
- 转真人转到哪个号码？没人接的时候怎么办？

参考

- OpenAI Realtime conversations: https://developers.openai.com/api/docs/guides/realtime-conversations
- OpenAI Realtime SIP: https://developers.openai.com/api/docs/guides/realtime-sip
- Twilio Elastic SIP Trunking × OpenAI: https://www.twilio.com/en-us/blog/developers/tutorials/product/openai-realtime-api-elastic-sip-trunking
- AI 前台架构概览 (freeCodeCamp): https://www.freecodecamp.org/news/how-ai-receptionists-work-the-architecture-behind-ai-phone-agents/